Changelog

What changed in each version.

Not released yet

Added

- Sprites editor (align mode): the graphic workspace (zoom levels, rulers, axis cross, guide lines, backgrounds, and dragging the sprite to move its axis) and the left panel (sprite selector, name, group, index, axis, 1 px moves, decompress on load, transparency color, onion skin). Every change can be undone, and axis changes in a row merge into one. Sprites menu: Add (the Add sprites dialog with numbers from the file names, axis points, palette modes, cropping and sharing), Delete, Duplicate, Change image, Save image, Save image (aligned) with axis.txt, Resize (six filters, and it can scale the frames' collision boxes too), Crop image, Delete group, Rename group, Alignment tool, Split channels, Export (FFE and SprMaker2 lists with PNG and ACT files), Import (FFE), and Save as in the M.U.G.E.N Beta, 1.0 or 1.1 format, turning true color sprites into 8-bit. The organizer moves and sorts sprites and palettes, numbers used twice are reported in Issues, and the status bar shows the cursor position, guide lines and SFF version.
- Sprite files keep linked sprites and palettes through any change, and are written back byte for byte when nothing changed.

Fighter Factory bugs fixed here

- Split channels gives each channel sprite its own channel, Add's Force and Exchange palette modes and "Crop before axis" work, Decompress on load is saved, Crop image treats index 0 as see-through for every range, and the status bar shows both guide positions.

Version 0.4.0 (15 August 2026)

The Definitions, Commands and States editors are done (milestone M4), with Run in engine, the variable usage list and log, the Throw creator and the Offset viewer.

Added

- Audio library (ofsaudio, without Qt): sample buffers with WAV conversion, reading WAV (PCM, float, A-law, mu-law, IMA and Microsoft ADPCM, MPEG), Ogg Vorbis, FLAC and MP3 (recognized by their content), writing WAV, Ogg Vorbis and FLAC, resampling, the waveform editing of the Sounds editor, and waveform peak summaries. Sound playback with a cursor, pause and loops, using miniaudio (AudioEngine, SoundPlayer).
- Project > Run in engine: the project is copied into the engine folder of the run configuration you pick (characters to chars/ofs_N, stages to stages/ofs_N.def, screenpacks to data/ofs_N) with your unsaved changes. The engine is started there, its mugen.log shows in Engine Output when it closes, and the copy is removed. Files that were already in the engine folder are never written over or deleted. Run configurations are saved in mugen_config.xml, the same way Fighter Factory saves them.

- States > View variable usage list: the code block tree shows, for each of the 60 integer and 40 float variables, the StateDefs that set it (click one to show it at the top of the editor). It follows your changes and remembers which entries you opened. Float variables and fvar(n) := assignments are listed as floats, and system variables are left out (Fighter Factory mixed them up).
- States > Insert variable usage log: the same list as a comment block at the cursor, by StateDef number, name or both, with every slot or one StateDef per line if you want.
- States > Throw creator: makes the states of a throw (attempt, throw with one TargetBind per element, held and thrown opponent) from the project's animations, shows both players on each element of the throw (the opponent drawn with the sprites of any open project, showing the thrown animation's frame at that moment), and puts the code in at the cursor. Add animation, Insert frame, Remove current frame and the Group, Index and flip fields edit the thrown animation in the AIR file directly. The text comes from data/engines/mugen/scripts/throw.lua, which you can edit. Different from Fighter Factory, the attempt uses the Start animation and the throw uses the Throwing animation, Insert frame splits at the element shown, Add animation uses its Time and refuses numbers already used, and the preview stays visible at big screen sizes.
- States > Offset viewer: a frame from one of the project's animations next to a frame from an animation of any open project, moved by an X and Y offset and flipped if you want, to work out bind and helper offsets. The viewer doesn't change anything in the project.
- New, Open and Save as in the Definitions, Commands and States editors (and the general Editor actions). New starts a file without a name in a new file tab that asks for a name when you save it, Open adds a file as a file tab starting in the editor's last folder, and Save as saves the current file under a new name.
- Engine plugins can run Lua scripts that return text (Script::callFunction) and open their own tool dialogs from their actions (Engine::toolActions, Engine::runTool).

Fixed

- Save as: the file tab, its checks, its code tree and the project's file list now follow the file to its new name (they lost track of it before).

Version 0.3.0 (1 August 2026)

The first preview: the main window, the file formats library and the code editors (milestones M0 to M3).

Added

- Project setup: CMake build with presets for Windows (llvm-mingw, MSVC), Linux and macOS.
- The core library (ofscore), the file formats library (ofsformats), the shared Lua library (ofslua) and the M.U.G.E.N engine plugin.
- Engine plugins are found and loaded, with a version check.
- Unit and GUI tests (Catch2, QtTest) and automatic builds.
- Settings files (config.xml, shortcuts.xml, history.xml) with defaults and user changes.
- All the actions of the program and the M.U.G.E.N editors, with shortcuts for each editor and menu items that show their keys.

- Lua presets for the interface (Classic, Clean, Neo, Ultimate) that build the toolbars and editor panels.
- Main window: menus, Main and Extra toolbars, project tabs, the welcome page with recent projects, left and right panels, file tabs, bottom panels and the status bar. M.U.G.E.N projects of every type open.
- Options dialog (General, Interface with the keyboard shortcut editor, Messages, Colors, Graphics, Code and the M.U.G.E.N tab), About dialog, Calculator and Clipboard container. Settings can be exported to one file, settings from an older install can be imported, and default settings come with the program.
- Themes: the Dark (default), Light, Classic and Ultimate interface styles, the six color sets, and the Nuvola, Flat-Dark, Flat-Light, Classic and Ultimate icon sets with their M.U.G.E.N extras, made from openly licensed icons by tools/iconsets/build_iconsets.py.
- File formats library (ofsformats, without Qt):
    - SFF sprite files v1, v2.0 and v2.01: reading, writing, saving files that weren't changed byte for byte, changing between v1 and v2, linked sprites and palettes, and choosing the compression for each sprite.
    - Sprite codecs RLE8, RLE5, LZ5, PCX and PNG (with exact palette indices).
    - SND sound files, WAV files (PCM 8, 16, 24 and 32-bit and float, unknown chunks kept) and FNT v1 bitmap fonts.
    - Palette files: ACT (M.U.G.E.N's backwards order and the normal one), RIFF PAL, JASC-PAL, ACO (reading) and palettes from PCX and PNG images.
    - Image tools: crop to the content, flips, nearest and bilinear resize, Scale2x, and fitting RGBA images to a palette.
- Images, palettes and tool input files for the editors (ofsformats): reading and writing GIF (all frames, local palettes, transparency, disposal, delays, looping), reading and writing BMP (1, 4 and 8-bit with RLE, 16, 24 and 32-bit), ASE palettes, writing ACO, palettes from GIF and BMP images, the six filters of the Resize dialog (nearest neighbor, bilinear, EPX/Scale3x and hqx through the hqx library), and reading, writing and exporting SprMaker2 .def files and SndMaker lists from SFF and SND files, checked against Elecbyte's sprmake2 and sndmaker.
- ofs-sffdump, a command line tool to look inside SFF files, extract and convert them, and list SND files.
- The byte layouts and odd cases of every format, in the File formats document.
- Tests on real M.U.G.E.N files on your computer and checks against Elecbyte's sff2png and sprmake2 (optional, turned on with OFS_CORPUS_KFM and OFS_CORPUS_MUGEN).
- M.U.G.E.N language data for the code editors: the syntax database (every file type, state controller and trigger from the DOS versions to 1.1), completion lists, snippets, state controller templates, PalFX presets and reference pages for controllers and triggers, with a checker and tests on real files.
- Code editors: line numbers, folding sections, highlighting of the current line, block and brackets, syntax highlighting from the syntax database, comment on and off, zoom, completion lists, snippets with input fields, and reference pages for controllers and triggers on Ctrl+click.
- Definitions, commands and states are checked as you type. Problems are underlined, listed in the code toolbar and the Issues panel, counted in the status bar, and the previous and next error

and deprecation commands jump to them. ofs-validate runs the same checks from the command line.

- Text encodings: the file's encoding is detected, Edit > Encoding (also in the status bar), the default charset option, and changing tabs to spaces and back on open and save.
- Search: the quick search bar (case, regular expressions with %1 captures, backwards and wrap around), the Find and Replace dialog for several lines, and Go to line.
- Advanced Search in all projects, the current project, the current file, the open files or a folder, with file and exclude patterns, results grouped by file, Search & Replace with check boxes, and the Lazy syntax fix (keywords written with their first letters are spelled out).
- Edit > Code > Auto-indent selection and Auto-format selection (lined up = signs and comments, respaced expressions), and the code editor's right click menu with the Code submenu.
- Code snippet editor (fields, preview, save, save for translation) and the code block trees of the Definitions, Commands and States editors, with Current line navigation.
- States editor: the list of state controller templates with New, Edit and Delete.
- Project explorer: a double-click opens a file in its editor.